

R Programming Tutorial

#03

Descriptive Statistics

#03

DESCRIPTIVE STATISTICS WITH R

Presenting Categorical Data

Frequency Distribution

- To organize the data using `table()` function in R
- Example (refer #lectureTWO, slide 7):

	A	B	C	D	E	F
1	Staff	Position	Blood Type	Weight	Height	Qualification
2	1	Senior Lecturer	A	60	172	PhD
3	2	Lecturer	B	55	164	Master
4	3	Professor	O	65	163	PhD
5	4	Associate Professor	AB	70	166	PhD
6	5	Associate Professor	O	61	167	PhD
7	6	Senior Lecturer	O	58	162	PhD
8	7	Senior Lecturer	B	48	172	PhD
9	8	Lecturer	A	68	161	Master
10	9	Lecturer	A	55	169	Master
11	10	Associate Professor	AB	62	158	PhD
12	11	Professor	O	58	175	PhD
13	12	Senior Lecturer	O	58	156	PhD
14	13	Senior Lecturer	B	48	154	PhD
15	14	Senior Lecturer	O	58	169	PhD
16	15	Senior Lecturer	B	48	160	PhD
17	16	Lecturer	A	68	178	Master
18	17	Lecturer	A	55	156	Master
19	18	Tutor	A	55	174	Master
20	19	Professor	O	58	167	PhD
21	20	Senior Lecturer	O	58	154	PhD
22	21	Senior Lecturer	B	48	158	PhD

Academic staff info: 140 staffs

Steps to analyze the data:

1. Import the data into R
2. Extract frequency distribution for desired information
3. Plot the frequency distribution in a bar plot

Presenting Categorical Data

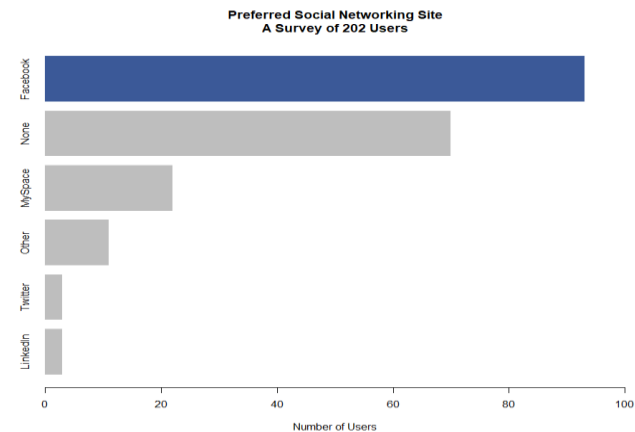
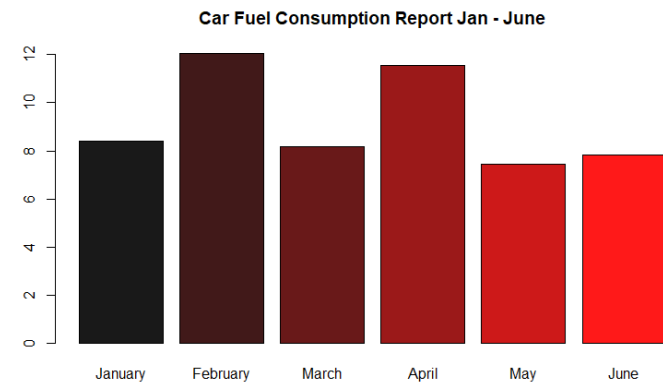
Frequency Distribution

- **STEP 1:** At the menu bar of Rstudio, go to:
File ? Import Data ? From Excel select the data file and import into the R environment.
- **STEP 2:** Extract the frequency distribution information using following command:
staffPosition.freq <- table(Data[,2])
- **STEP 3:** Plot the frequency distribution using bar plot
barplot(staffPosition.freq)

	A	B	C	
1	Staff	Position	Blood Type	We
2	1	Senior Lecturer	A	6
3	2	Lecturer	B	5
4	3	Professor	O	6
5	4	Associate Professor	AB	7
6	5	Associate Professor	O	6
7	6	Senior Lecturer	O	5
8	7	Senior Lecturer	B	4
9	8	Lecturer	A	6
10	9	Lecturer	A	5
11	10	Associate Professor	AB	6
12	11	Professor	O	5
13	12	Senior Lecturer	O	5
14	13	Senior Lecturer	B	4
15	14	Senior Lecturer	O	5
16	15	Senior Lecturer	B	4
17	16	Lecturer	A	6
18	17	Lecturer	A	5
19	18	Tutor	A	5
20	19	Professor	O	5
21	20	Senior Lecturer	O	5
22	21	Senior Lecturer	B	4

Bar Plot in R

- In R, bar chart / barplot is plotted using **barplot()** function.
- barplot()** is a built-in function
- Example: **bar(variable, col=color, border=NA, xlim=range, main=title, xlab=x-axis, ylab=y-axis name)**



Presenting Data using barplot()

- To plot the academic staff data into a bar plot, use following command:

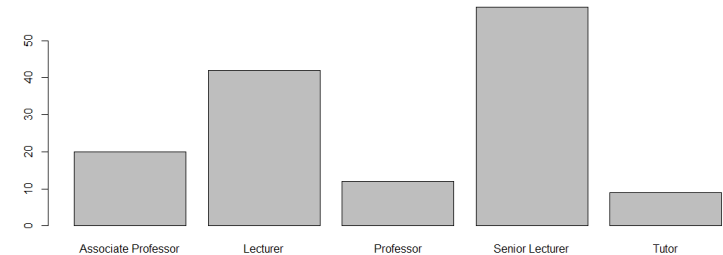
```
barplot(staffPosition.freq)
```

- What if we want to reorder it?
- Using following command in R we can reorder the data:

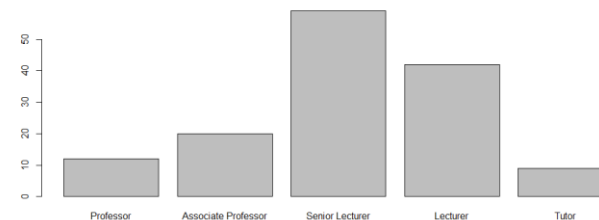
```
staffPosition = staffPosition.freq[c(3,1,4,2,5)]
```

- Then replot again for the new ordered data:

```
barplot(staffPosition)
```



Reorder based on the index



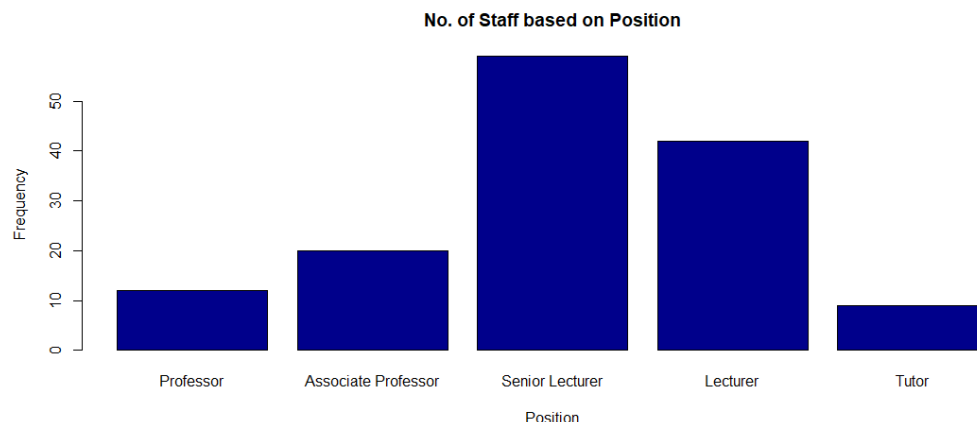
Settings in barplot()

- Settings of bar plot available in R include:

- Color
- Bar orientation
- Range
- Title, labels

- For example:

```
barplot(staffPosition, col="darkblue", xlab="Position",  
ylab="Frequency", main="No. of Staff based on Position")
```



Presenting Data using pie()

- Example: let say we have a set of data of number of students for 2017/2018 Intake as follow:

- Let say we import the data and save it into a variable name

Data2

- After import the data in R, we can add the column names using:

```
colnames(Data2) <- c("X1", "X2")
```

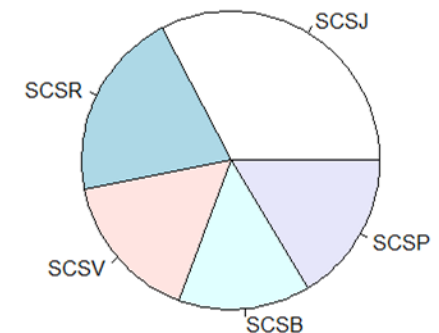
- Then, a pie chart can be plotted using following command:

```
pie(Data2$X2, Data2$X1)
```

Data to be plotted

Labels

Course	No. of students
SCSJ	80
SCSR	50
SCSV	40
SCSB	35
SCSP	40



More with pie chart in R

- 3D pie chart is available with the use of package *plotrix*

```
install.packages("plotrix")  
library("plotrix")
```

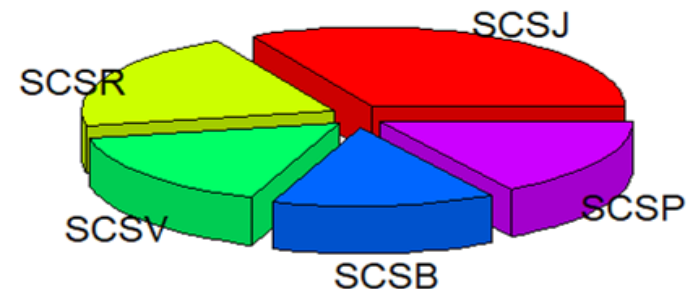
- After install the package, use the function **pie3D()**
- Example using the student data:

```
pie3D(Data2$X2, labels=Data2$X1,  
      main="No. of Students For 2017/2018 Intake",  
      explode=0.1)
```

"explode"
parameter for the
separation
between each
slice

The function of **pie3D** required package named "**plotrix**" to work. Please install the package before you try this.

No. of Students For 2017/2018 Intake



Relative Frequency Distribution

- Refer back to the academic staff data in Slide 3.
- Let say we already have the frequency distribution in the variable ***staffPosition.freq***
- We can calculate the relative frequency distribution simply with following command

```
staffPosition.rfd <- staffPosition.freq/140
```

Position	Number of Staff (frequency)
Professor	12
Associate Professor	20
Senior Lecturer	59
Lecturer	40
Tutor	9
Total	140

- In R, arithmetic executed for a list variable will apply to each of the element.

```
> staffPosition.rfd <- staffPosition.freq/140
> staffPosition.rfd
```

```
      Professor Associate Professor      Senior Lecturer      Lecturer      Tutor
0.08571429    0.14285714    0.42142857    0.28571429    0.06428571
> |
```

- Then plot the relative frequency distribution into a bar plot

Presenting Data using stem and leaf

- Let say we have a list of numbers in a variable named **num**:
12, 13, 21, 27, 33, 34, 35, 37, 40, 40, 41
- In R in order to draw a stem-and-leaf plot, we can use the `stem()` function.

`stem(num)`

```
> num <- c(12, 13, 21, 27, 33, 34, 35, 37, 40, 40, 41)
> stem(num)

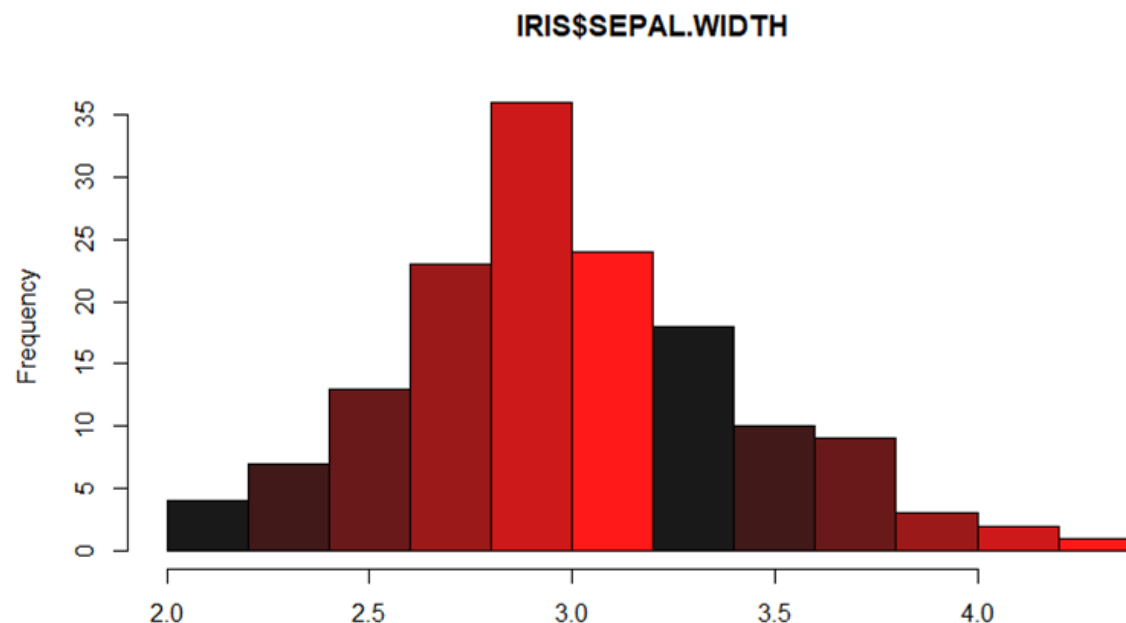
The decimal point is 1 digit(s) to the right of the |

 1 | 23
 2 | 17
 3 | 3457
 4 | 001
```

- x:** Please specify the data on which you want to draw the Stem and Leaf Plot. Here, you have to use the numeric vector, or a list containing numeric vector.
- scale:** Please specify the scale you want to use for your plot.
- width:** It is optional but you can use this to specify the desired width of a plot. By default it is 80
- atom:** It is a tolerance

Histogram in R

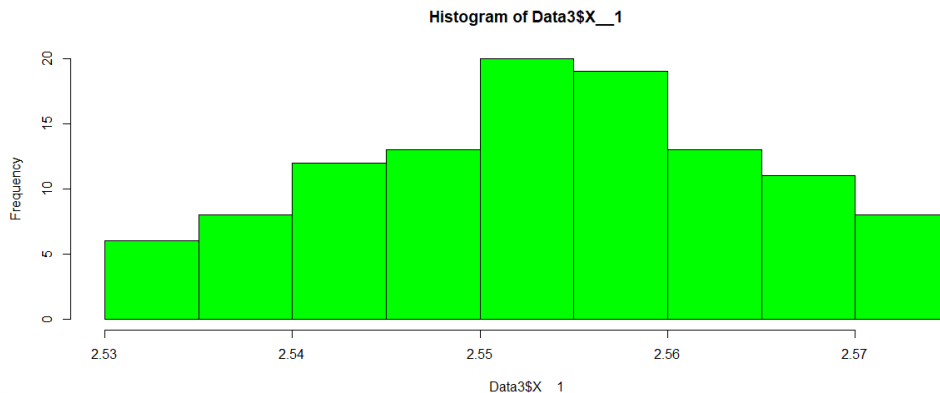
- In R, histogram is plotted using **hist()** function.
 - Example: **hist(variable, col=color, border=NA, xlim=range, main=title, xlab=x-axis, ylab=y-axis name)**



Presenting data in histogram

- Let say we have a group of data and imported into R in a variable named **Data3**
- Add column name using: `colnames(Data3) <- c("X1")`
- To plot a histogram out of the data, we can use following:
`hist(Data3$X1, col="green")`
- We can save the output by assigning the command above into a variable.

```
output_h = hist(Data3$X1, col="green");
```



2.559	2.556	2.566	2.546	2.561	2.570	2.546
2.565	2.543	2.538	2.560	2.560	2.545	2.551
2.568	2.546	2.555	2.551	2.554	2.574	2.568
2.572	2.550	2.556	2.551	2.561	2.560	2.564
2.567	2.560	2.551	2.562	2.542	2.549	2.561
2.556	2.550	2.561	2.558	2.556	2.559	2.557
2.532	2.575	2.551	2.550	2.559	2.565	2.552
2.560	2.534	2.547	2.569	2.559	2.549	2.544
2.550	2.552	2.536	2.570	2.564	2.553	2.558
2.538	2.564	2.552	2.543	2.562	2.571	2.553
2.539	2.569	2.552	2.536	2.537	2.532	2.552
2.575 (h)	2.545	2.551	2.547	2.537	2.547	2.533
2.538	2.571	2.545	2.545	2.556	2.543	2.551
2.569	2.559	2.534	2.561	2.567	2.572	2.558
2.542	2.574	2.570	2.542	2.552	2.551	2.553
2.546	2.531 (l)	2.563	2.554	2.544		

Presenting data in histogram

- This variable holds the information calculated by R during the construction of the histogram.

```

> hist(Data3$X__1, col="green")
> output_h = hist(Data3$X__1, col="green")
> output_h
$breaks
 [1] 2.530 2.535 2.540 2.545 2.550 2.555 2.560 2.565 2.570 2.575

$counts
 [1]  6  8 12 13 20 19 13 11  8

$density
 [1] 10.90909 14.54545 21.81818 23.63636 36.36364 34.54545 23.63636 20.00000 14.54545

$mids
 [1] 2.5325 2.5375 2.5425 2.5475 2.5525 2.5575 2.5625 2.5675 2.5725

$xname
 [1] "Data3$X__1"

$equidist
 [1] TRUE

attr(,"class")
 [1] "histogram"
> |
  
```

Interval calculated by R

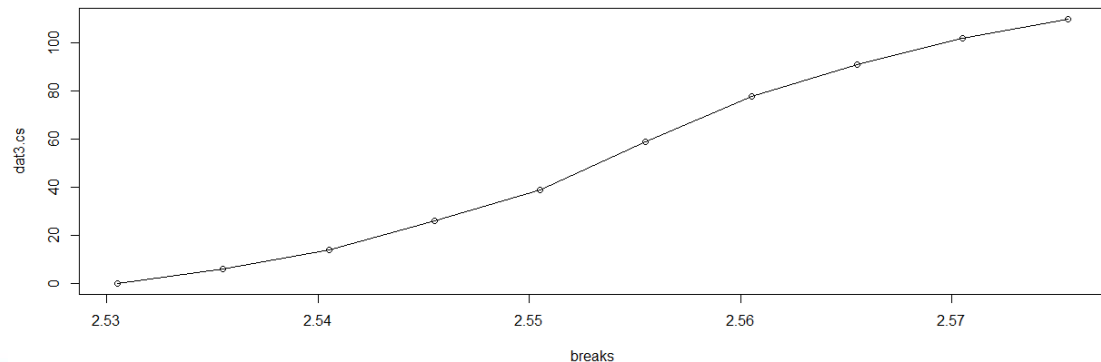
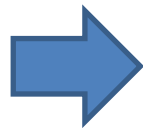
Grouped frequency

Mid points

Frequency distribution of grouped data

- Consider the data used in histogram plotting.
- We can group the data based on desired breaks.
- Generally, follow steps below
 1. Extract the data (if it is in dataframe) into a list or vector
 2. Define the breaks (a.k.a interval)
 3. Group the data based on the breaks
 4. Calculated cumulative frequency
 5. Plot the result.

2.559	2.556	2.566	2.546	2.561	2.570	2.546
2.565	2.543	2.538	2.560	2.560	2.545	2.551
2.568	2.546	2.555	2.551	2.554	2.574	2.568
2.572	2.550	2.556	2.551	2.561	2.560	2.564
2.567	2.560	2.551	2.562	2.542	2.549	2.561
2.556	2.550	2.561	2.558	2.556	2.559	2.557
2.532	2.575	2.551	2.550	2.559	2.565	2.552
2.560	2.534	2.547	2.569	2.559	2.549	2.544
2.550	2.552	2.536	2.570	2.564	2.553	2.558
2.538	2.564	2.552	2.543	2.562	2.571	2.553
2.539	2.569	2.552	2.536	2.537	2.532	2.552
2.575 (b)	2.545	2.551	2.547	2.537	2.547	2.533
2.538	2.571	2.545	2.545	2.556	2.543	2.551
2.569	2.559	2.534	2.561	2.567	2.572	2.558
2.542	2.574	2.570	2.542	2.552	2.551	2.553
2.546	2.531 (b)	2.563	2.554	2.544		



Frequency distribution of grouped data

- STEP 1: Extract the data (if it is in dataframe) into a list or vector

```
Dat3 <- Data3$X1
```

Extract the info and assign into Dat3 which is a vector

- STEP 2: Define the breaks (a.k.a interval)

```
breaks <- seq(2.5305, 2.5375, 0.005)
```

(low limit, up limit, interval)

- STEP 3: Group the data based on the breaks

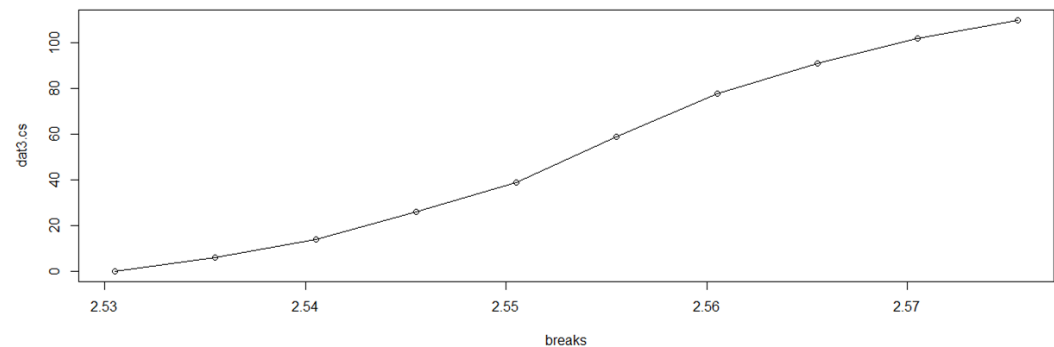
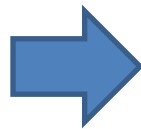
```
Dat3.cut <- cut(Dat3, breaks)
```

Categorize each data based on the breaks

```
Dat3.freq <- table(Dat3.cut)
```

Group the similar breaks and get the frequency

2.559	2.556	2.566	2.546	2.561	2.570	2.546
2.565	2.543	2.538	2.560	2.545	2.551	
2.568	2.546	2.555	2.551	2.554	2.574	2.568
2.572	2.550	2.556	2.551	2.561	2.560	2.564
2.567	2.560	2.551	2.562	2.542	2.549	2.561
2.556	2.550	2.561	2.558	2.556	2.559	2.557
2.532	2.575	2.551	2.550	2.559	2.565	2.552
2.560	2.534	2.547	2.569	2.559	2.549	2.544
2.550	2.552	2.536	2.570	2.564	2.553	2.558
2.538	2.564	2.552	2.543	2.562	2.571	2.553
2.539	2.569	2.552	2.536	2.537	2.532	2.552
2.575 (h)	2.545	2.551	2.547	2.537	2.547	2.533
2.538	2.571	2.545	2.545	2.556	2.543	2.551
2.569	2.559	2.534	2.561	2.567	2.572	2.558
2.542	2.574	2.570	2.542	2.552	2.551	2.553
2.546	2.531 (l)	2.563	2.554	2.544		



Frequency distribution of grouped data

- STEP 4: Calculated cumulative frequency using **cumsum()**

```
Dat3.cs <- c(0, cumsum(Dat3.freq))
```

Calculate cumulative freq

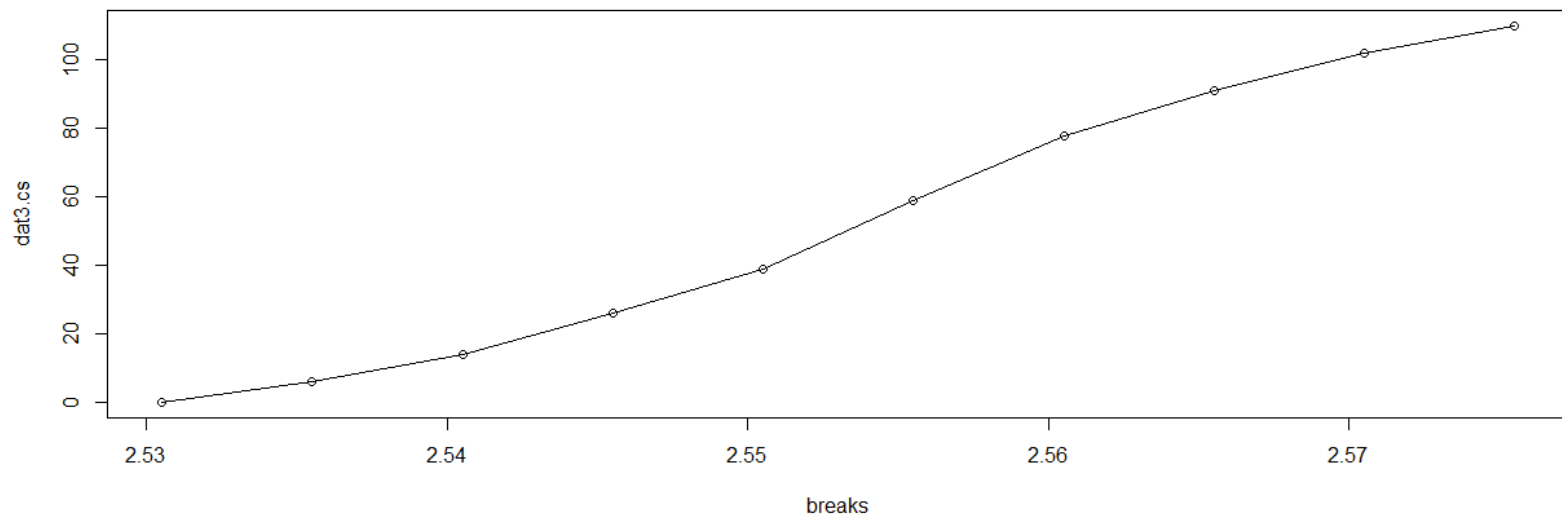
- STEP 5: Plot the results using **plot()** function

```
plot(breaks, Dat3.cs)
```

Plot a graph

```
lines(breaks, Dat3.cs)
```

Link the dots with line

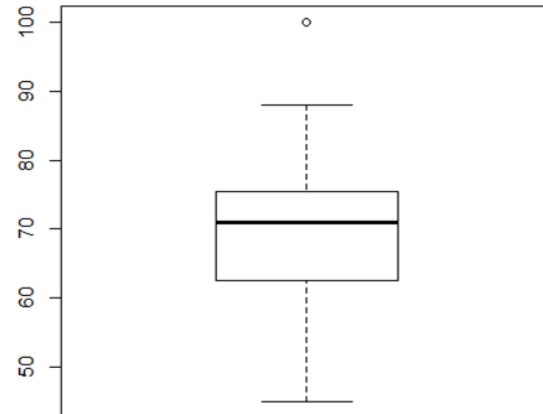


Presenting Data in boxplot()

- Let say we have a set of data:

```
45 48 50 54 57 60 60 62 63 63 64 65 67 68 69  
71 71 72 72 74 74 75 75 76 80 83 84 88 100 100  
72 100
```

- First we have to make sure the data we have is in data frame in R. Let say we save this in a variable named **num2**
- Then to plot a boxplot, simply use following command:
boxplot(num2)
- We can also save the output of the boxplot function into a variable, let say **output**
output = boxplot(num2)



More in boxplot in R

- By assigning the output of a boxplot into a variable, information calculated by R can be revealed.

A list is returned which has

- stats - having the position of the upper/lower extremes of the whiskers and box along with the median
- n - the number of observation the boxplot is drawn with (notice that NA's are not taken into account)
- conf - upper/lower extremes of the notch, out-value of the outliers
- group - a vector of the same length as out whose elements indicate to which group the outlier belongs and
- names - a vector of names for the groups.

```
> output <- boxplot(num2)
> output
$stats
      [,1]
[1,] 45.0
[2,] 62.5
[3,] 71.0
[4,] 75.5
[5,] 88.0

$n
[1] 31

$conf
      [,1]
[1,] 67.31091
[2,] 74.68909

$out
[1] 100 100 100

$group
[1] 1 1 1

$names
[1] "bdata"

> |
```

Descriptive Statistics (Mean)

- Mean can be calculated using **mean()** function.
- For simple data such as a sequence of numbers. Let say as following:

```
num3 = c(20, 30, 40, 50, 80, 100)
```

- The mean can be calculated simply with following command:

```
mean(num3)
```

```
> num3 = c(20, 30, 40, 50, 80, 100)
>
> mean(num3)
[1] 53.33333
> |
```

Descriptive Statistics (Median)

- Median can be calculated using **median()** function.
- Refer back to the same data:

```
num3 = c(20, 30, 40, 50, 80, 100)
```

- The median can be calculated simply with following command:

```
median(num3)
```

```
> num3 = c(20, 30, 40, 50, 80, 100)
>
> median(num3)
[1] 45
> |
```

Descriptive Statistics (Mode)

- R **does not** have a standard in-built function to calculate mode. So we create a user function to calculate mode of a data set in R.
- Assume we have a list of numbers as follows:

```
num4 = c(20, 30, 40, 50, 20, 80, 100)
```
- Mode can be calculated using following steps:
- STEP 1: get the available unique number using **unique()**

```
unique.value <- unique(num4)
```
- STEP 2: find occurrence of unique number using **match()**

```
unique.match <- match(num4, unique.value)
```

Descriptive Statistics (Mode)

- STEP 3: find the frequency of each unique number using **tabulate()**

```
unique.freq <- tabulate(unique.match)
```

- STEP 4: find the index which has the highest frequency using **which.max()**

```
unique.max <- which.max(unique.freq)
```

- STEP 5: get the value of the mode by using the index identified in STEP 4

```
num4_mode <- unique.value[unique.max]
```

```
> unique.value <- unique(num4)
> unique.value
[1] 20 30 40 50 80 100
> unique.match <- match(num4, unique.value)
> unique.match
[1] 1 1 2 3 4 5 6
> unique.freq <- tabulate(unique.match)
> unique.freq
[1] 2 1 1 1 1 1
> unique.max <- which.max(unique.freq)
> unique.max
[1] 1
> num4_mode <- unique.value[unique.max]
> num4_mode
[1] 20
> |
```

Descriptive Statistics (Quartile)

- In R, information for quartile can be obtained using `quantile()` function.
- Let say we have a list of data as follows:

```
num5 = c(12, 4, 6, 11, 9, 15, 20, 18, 25, 30)
```

- The quartile information can be obtained via following:

```
quantile(num5, type=2)
```

- There are several types of setting for the function that will generate different results

```
> quantile(num5, type=2)
 0%   25%   50%   75%  100%
4.0   9.0  13.5  20.0  30.0
> |
```

- Source: [https://tolstoy.newcastle.edu.au/R/e17/help/att-1067/Quartiles in R.pdf](https://tolstoy.newcastle.edu.au/R/e17/help/att-1067/Quartiles%20in%20R.pdf)

Descriptive Statistics (variance / std dev)

- In R, variance can be calculated using the **var()** function.
- Using the same data as previous slide, variance can be obtained using the following command:

```
num5_variance <- var(num5)
```

- While for standard deviation, can be obtained using **sd()** in R
- For example:

```
num5_sd <- sd(num5)
```

```
> num5_variance <- var(num5)
> num5
[1] 12  4  6 11  9 15 20 18 25 30
> num5_variance
[1] 69.11111
> |
```

```
> num5_sd <- sd(num5)
> num5
[1] 12  4  6 11  9 15 20 18 25 30
> num5_sd
[1] 8.313309
> |
```

Skewness in R

- In R, skewness can be measured using **skewness()** function in the package named “moments”

- Let say we have a set of data as follows:

```
time <- c(19.09, 19.55, 17.89, 17.73, 25.15, 27.27, 25.24, 21.05, 21.65,  
20.92, 22.61, 15.71, 22.04, 22.60, 24.25)
```

- The skewness of the data can be measured using following command:

```
skewness(time)
```

This required package named “**moments**” to work. Please install the package before you try this.

```
> time <- c(19.09, 19.55, 17.89, 17.73, 25.15, 27.27, 25.24,  
21.05, 21.65, 20.92, 22.61, 15.71, 22.04, 22.60, 24.25)  
>  
> skewness(time)  
[1] -0.01565162  
> |
```

Kurtosis in R

- In R, kurtosis can be measured using **kurtosis()** function in the package named “moments” as same as previous.
- Let say we use the same data as previous slide:

```
time <- c(19.09, 19.55, 17.89, 17.73, 25.15, 27.27, 25.24, 21.05, 21.65,  
20.92, 22.61, 15.71, 22.04, 22.60, 24.25)
```

- The kurtosis of the data can be measured using following command:

```
kurtosis(time)
```

This required package named “**moments**” to work. Please install the package before you try this.

```
> time <- c(19.09, 19.55, 17.89, 17.73, 25.15, 27.27, 25.24,  
21.05, 21.65, 20.92, 22.61, 15.71, 22.04, 22.60, 24.25)  
>  
> kurtosis(time)  
[1] 2.301051  
> |
```